Programming and Computational Techniques • Python • Extra material for Unit 1

Python 2 vs Python 3

Recognising and updating old code

David Ferro Costas • Theoretical Chemistry and Computational Modelling • 2026/27

How to use this document. Everything in the course is Python 3. This extra material is for the moment you find older code (a script from a colleague, an example in an old book, an answer online) and need to understand it or make it run. Python 3 examples appear in **blue** boxes and have all been run; Python 2 examples appear in **brown** boxes, followed by what Python 2 printed.

Contents

1 Why this still matters . . . 1
2 Quick reference . . . 2
3 Printing: from statement to function . . . 2
4 Division: the silent difference . . . 3
5 Reading input . . . 4
6 Lists that became lazy: `range`, `zip`, `map`, `filter` . . . 4
7 Dictionaries . . . 5
8 Rounding . . . 6
9 String formatting . . . 7
10 Text, bytes and encodings . . . 7
11 Other differences you may meet . . . 8
12 A worked example . . . 8
13 Checklist for translating old code . . . 10
14 Exercise: spot the Python 2 . . . 10
15 Further reading . . . 11
Solution to the exercise . . . 11

1 Why this still matters

Python 3.0 was released in December 2008. It was deliberately *not* backwards compatible: its designers took the opportunity to fix several old decisions in the language, knowing that this would break existing programs. For more than a decade both versions lived side by side. Python 2.7, the last version of Python 2, reached its official end of life on 1 January 2020 (the final release, 2.7.18, came out in April 2020). It receives no fixes of any kind, and NumPy, SciPy, Matplotlib and practically every scientific library have long since dropped it.

Even so, a lot of scientific code was written in Python 2 and never updated: utility scripts passed from student to student, analysis tools in old repositories, examples in books, answers in forums. Sooner or later you will meet some. You need to be able to

1. **recognise** it, so that you do not copy it blindly; and
2. **translate** it to Python 3 *without changing what it computes*.

Most differences make old code fail loudly in Python 3 (a `SyntaxError` or a `NameError` on the first run), which is annoying but harmless. A few change results *silently*. Those are the ones this document insists on, especially integer division (Section 4).

Which Python am I running? From a terminal, `python3 --version`. Inside a program, `sys.version` gives the full version string and `sys.version_info` can be compared:

Python 3 (interactive)

```
>>> import sys
>>> sys.version_info >= (3, 10)
True
```

2 Quick reference

	Python 2	Python 3	Sec.
Printing	`print "E =", E` (a statement)	`print("E =", E)` (a function)	3
Division	`7 / 2` → `3`	`7 / 2` → `3.5`; `7 // 2` → `3`	4
Reading text	`raw_input()`; `input()` evaluated the text	`input()`, always a string	5
`range(5)`	a list (`xrange` was lazy)	a lazy `range` object	6
`zip`, `map`, `filter`	lists	single-use iterators	6
`d.keys()`, ...	lists; also `d.has_key(k)`, `d.iteritems()`	views; `k in d`, `d.items()`	7
Rounding	`round(2.5)` → `3.0`	`round(2.5)` → `2`	8
Formatting	`"%.3f" % x`	`f"{x:.3f}"` (`%` still works)	9
Text	`str` = bytes; `unicode` for text	`str` = Unicode text; `bytes` for bytes	10
Exceptions	`except IOError, e:`	`except OSError as e:`	11
Mixed comparisons	`1 < "a"` → `True`	`TypeError`	11
Integers	`int` and `long` (`10L`)	one `int` type	11
Octal literals	`0755`	`0o755`	11
Not equal	`<>` or `!=`	only `!=`	11

The quickest way to tell: if you see `print` without parentheses, `raw_input`, `xrange`, `.has_key(` or `u"..."` strings, it is Python 2.

3 Printing: from statement to function

In Python 2, `print` was a *statement* of the language, like `if` or `for`. In Python 3 it is an ordinary *function*: it needs parentheses, and its behaviour is controlled with keyword arguments.

Python 2

```
print "E =", E
print "no new line",
print >>f, "to a file"
print
```

Python 3

```
print("E =", E)
print("no new line", end=" ")
print("to a file", file=f)
print()
```

A trailing comma in Python 2 suppressed the new line; in Python 3 that is `end=" "`. The odd `print >>f` syntax for writing to a file became `file=f`. Python 3 recognises the old form and tells you exactly what is wrong:

```
$ python3 old_script.py
  File "old_script.py", line 9
    print "Mean number of H-bonds: %.2f" % mean
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

Watch out! The reverse direction is subtler. In Python 2, `print("E =", E)` is the old statement applied to a *tuple*, so it prints `('E =', -76.0268)` instead of `E = -76.0268`. Some Python 2 code starts with `from __future__ import print_function` precisely to get the Python 3 behaviour.

4 Division: the silent difference

This is the most important section of this document, because the change does not produce any error.

- In **Python 2**, `/` between two *integers* was *floor division*: the result was rounded down to an integer. Only if at least one operand was a float did it perform true division.
- In **Python 3**, `/` *always* performs true division and returns a float. Floor division has its own operator, `//`, which exists in both versions.

Expression	Python 2	Python 3
`7 / 2`	`3`	`3.5`
`7.0 / 2`	`3.5`	`3.5`
`-7 / 2`	`-4`	`-3.5`
`7 // 2`	`3`	`3`
`1 / 2 * m * v**2`	`0`	`0.5 * m * v**2`

4.1 Python 2 code that was silently wrong

Converting a temperature from Celsius to Fahrenheit with the textbook formula:

Python 2

```
T_C = 100
T_F = 9/5 * T_C + 32
print T_F
```

```
132
```

Python 3

```
T_C = 100
T_F = 9/5 * T_C + 32
print(T_F)
```

```
212.0
```

In Python 2, `9/5` is `1`, so the program prints 132 instead of 212, with no warning. Programmers who knew this wrote `9.0/5`, `1.0*n/m` or `float(n)/m` everywhere; when you see those, you are almost certainly reading Python 2 code (in Python 3 they are harmless but unnecessary). Code that forgot them may have produced wrong results for years: Section 12 shows a realistic case.

4.2 Python 2 code that relied on integer division

The opposite problem appears when old code *wanted* an integer, typically to use it as an index:

Python 2

```
middle = atoms[len(atoms)/2]
```

In Python 3, `len(atoms)/2` is a float, and floats cannot be used as indices:

Python 3

```
middle = atoms[len(atoms)/2]
```

```
TypeError: list indices must be integers or slices, not float
```

The fix is to say what you mean: `atoms[len(atoms) // 2]`.

> **Rule when translating.** Look at *every* `/` in the old code and decide what it was doing with the types it actually received. If it was a real division, keep `/` (and drop any `float()` or `1.0*` tricks). If it needed an integer, change it to `//`. No automatic tool can make this decision for you, because it depends on the types of the operands at run time.

You may also find `from __future__ import division` at the top of Python 2 files: it switched on the Python 3 behaviour, so in those files `/` already means true division.

5 Reading input

Python 2 had two functions to read from the keyboard:

- `raw_input()` returned what the user typed, as a string;
- `input()` *evaluated* what the user typed as Python code: typing `5` gave the integer 5, typing `[1, 2]` gave a list… and typing a malicious expression would run it.

Python 3 removed the dangerous version and renamed `raw_input()` to `input()`. It always returns a string, which you convert explicitly:

Python 2

```
T = float(raw_input("T (K): "))
n = input("n: ")        # evaluated!
```

Python 3

```
T = float(input("T (K): "))
n = int(input("n: "))
```

6 Lists that became lazy: `range`, `zip`, `map`, `filter`

Several built-in functions that returned *lists* in Python 2 return *lazy* objects in Python 3: they produce their values one at a time, when they are needed, instead of building the whole list in memory.

6.1 `range`

In Python 2, `range(10**9)` tried to build a list of a billion integers (several GB of memory). For loops, Python 2 programmers used `xrange`, which produced the numbers on demand. In Python 3, `range` *is* the lazy version and `xrange` no longer exists. A Python 3 `range` is still a proper sequence: it has a length, can be indexed and supports `in`, all without storing the numbers.

Python 3 (interactive)

```
>>> range(5)
range(0, 5)
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> r = range(0, 10**12, 2)      # no memory problem
>>> len(r), r[3], 10**11 in r
(500000000000, 6, True)
```

Translating: replace `xrange` by `range`. Wrap `range` in `list()` only if you really need a list (to modify it, for example).

6.2 `zip`, `map` and `filter`

These return single-use *iterators* in Python 3. Looping over them works exactly as before, but they cannot be indexed and are exhausted after one pass:

Python 3 (interactive)

```
>>> pairs = zip(["H", "O"], [1.008, 15.999])
>>> pairs[0]
TypeError: 'zip' object is not subscriptable
>>> list(pairs)
[('H', 1.008), ('O', 15.999)]
>>> list(pairs)                  # already used up
[]
```

In Python 2 code you may also find `itertools.izip`, `imap` and `ifilter`: they were the lazy versions and are simply `zip`, `map` and `filter` in Python 3. In new code, list comprehensions are usually clearer than `map` and `filter`.

7 Dictionaries

7.1 Keys, values and items are views

In Python 2, `d.keys()`, `d.values()` and `d.items()` returned new *lists*. In Python 3 they return *views*: lightweight windows onto the dictionary that do not copy anything and that follow its changes.

Python 3 (interactive)

```
>>> Z = {"H": 1, "He": 2}
>>> keys = Z.keys()
>>> keys
dict_keys(['H', 'He'])
>>> Z["Li"] = 3
>>> keys                         # the view sees the new key
dict_keys(['H', 'He', 'Li'])
>>> keys[0]
TypeError: 'dict_keys' object is not subscriptable
```

Views work in `for` loops, with `in` and with `len()`, which covers most uses. But a view is *not a list*, so two things that were very common in Python 2 code now raise an error:

- sorting the keys *in place* with `.sort()`: views have no `sort` method (`AttributeError`);
- taking a key *by position*, e.g. `d.keys()[0]`: views cannot be indexed (`TypeError`).

In Python 3, first turn the keys into a list. Looping over a dictionary gives its keys, so `sorted(d)` and `list(d)` are enough:

Python 2

```
names = d.keys()
names.sort()          # fails in Py3
first = d.keys()[0]   # fails in Py3
```

Python 3

```
names = sorted(d)     # sorted list
first = list(d)[0]    # first key
print(names, first)
```

```
['C', 'H', 'O'] O
```

7.2 Removed methods

Python 2	Python 3
`d.has_key(k)`	`k in d`
`d.iteritems()`, `d.iterkeys()`, `d.itervalues()`	`d.items()`, `d.keys()`, `d.values()`

Order. In Python 2, the order of the keys in a dictionary was arbitrary. Since Python 3.7, dictionaries keep the order in which keys were inserted. Old code often sorted the keys to get reproducible output; that still works, and is still a good idea when the order matters.

8 Rounding

Two things changed in `round()`:

1. **Ties.** Python 2 rounded halves *away from zero* (`round(2.5)` gave `3.0`, `round(-2.5)` gave `-3.0`). Python 3 rounds halves to the nearest *even* number, the default of the IEEE 754 standard ("banker's rounding"). When many values are rounded, always rounding halves up introduces a systematic bias; rounding to even does not.
2. **Type.** Python 2 always returned a float. Python 3 returns an integer when no number of decimals is given.

Python 3 (interactive)

```
>>> round(2.5), round(3.5), round(-2.5)
(2, 4, -2)
>>> round(2.5, 0)                # with decimals: a float
2.0
>>> format(0.125, ".2f")         # formatting rounds the same way
'0.12'
```

Not a bug. `round(2.675, 2)` gives `2.67` in *both* versions. The number 2.675 cannot be stored exactly in binary; the stored value is slightly below 2.675, so it rounds down. If you need decimal rounding rules (money, tabulated values), use the `decimal` module:

Python 3 (interactive)

```
>>> from decimal import Decimal, ROUND_HALF_UP
>>> Decimal("2.5").quantize(Decimal("1"), rounding=ROUND_HALF_UP)
Decimal('3')
```

9 String formatting

This is not really an incompatibility: the old % operator still works in Python 3. But you will see three styles, and it helps to recognise them as the same thing.

Python 3

```
mol, n, E = "H2O", 3, -76.0267656
print("%s: %d atoms, E = %.4f Eh" % (mol, n, E))        # % operator
print("{}: {} atoms, E = {:.4f} Eh".format(mol, n, E))  # str.format
print(f"{mol}: {n} atoms, E = {E:.4f} Eh")              # f-string
```

```
H2O: 3 atoms, E = -76.0268 Eh
H2O: 3 atoms, E = -76.0268 Eh
H2O: 3 atoms, E = -76.0268 Eh
```

The % style comes from C's `printf` and was the norm in Python 2; `str.format()` appeared in Python 2.6; f-strings in Python 3.6. Use f-strings in new code. Translating is mechanical, because the format codes are almost identical:

% style	f-string	Meaning
`"%d" % n`	`f"{n}"` or `f"{n:d}"`	integer
`"%5d" % n`	`f"{n:5d}"`	integer, width 5
`"%8.3f" % x`	`f"{x:8.3f}"`	width 8, 3 decimals
`"%.2e" % x`	`f"{x:.2e}"`	scientific notation
`"%-10s" % s`	`f"{s:<10}"`	left-aligned, width 10
`"%10s" % s`	`f"{s:>10}"`	right-aligned, width 10

Watch out! With %, several values must be given as a tuple, and the number of values must match the number of placeholders: `"%s" % (1, 2)` raises `TypeError: not all arguments converted during string formatting`. f-strings avoid this whole class of mistakes.

10 Text, bytes and encodings

This is the deepest change, although in everyday scientific scripts it is the one you notice least.

- In **Python 2**, the `str` type was a sequence of *bytes*. Text with characters outside ASCII (ñ, Å, °, Greek letters) needed a different type, `unicode`, written `u"..."`, and mixing the two types was a constant source of `UnicodeDecodeErrors`. A source file containing non-ASCII characters even needed a special first line, `# -*- coding: utf-8 -*-`.
- In **Python 3**, `str` is always *text* (a sequence of Unicode characters) and raw bytes have their own type, `bytes`, written `b"..."`. Source files are UTF-8 by default.

Python 3 (interactive)

```
>>> len("Å")                    # one character (in Python 2: 2 bytes)
1
>>> "Å".encode("utf-8")         # text -> bytes
b'\xc3\x85'
>>> b"\xc3\x85".decode("utf-8")  # bytes -> text
'Å'
```

For files the rule is simple: open text files in text mode (the default), and you get `str`; open binary files with `"rb"` or `"wb"`, and you get `bytes`. When a text file may contain non-ASCII

characters, say which encoding it uses by passing `encoding="utf-8"` to `open()`. Otherwise Python uses the system default, which is not the same on every computer.

In translated code, the `u` prefix of `u"..."` strings is harmless (Python 3 accepts and ignores it), `unicode(x)` becomes `str(x)`, and `basestring` becomes `str`.

11 Other differences you may meet

Python 2	Python 3	What Python 3 says about the old form
`except IOError, e:`	`except OSError as e:`	`SyntaxError: multiple exception types must be parenthesized`
`raise ValueError, "msg"`	`raise ValueError("msg")`	`SyntaxError: invalid syntax`
`1 <> 2`	`1 != 2`	`SyntaxError: ... Maybe you meant '!=' instead of '<>'?`
`0755` (octal)	`0o755`	`SyntaxError: leading zeros in decimal integer literals are not permitted; ...`
`10L`, `long(x)`	`10`, `int(x)`	`SyntaxError: invalid decimal literal`
`1 < "a"` → `True`	comparing different types is an error	`TypeError: '<' not supported between instances of 'int' and 'str'`
`file("data.txt")`	`open("data.txt")`	`NameError: name 'file' is not defined`
`string.split(line)`	`line.split()`	`AttributeError`: the old functions of the `string` module became string methods
`reduce(f, values)`	`functools.reduce(f, values)`	`NameError: name 'reduce' is not defined`

A few notes:

- Since Python 3.3, `IOError` is just another name for `OSError`, so `except IOError as e:` also works.
- Python 2 had two integer types, `int` and `long`, and converted automatically between them. Python 3 has a single `int` with arbitrary precision.
- Python 2 let you compare and sort objects of different types (numbers were "smaller" than strings). Python 3 refuses, which usually reveals a real bug, such as numbers read from a file that were never converted with `float()`.

12 A worked example

The following Python 2 script reads a file with the number of hydrogen bonds found in each frame of a molecular dynamics simulation, and reports the mean and the frames below it. The file `hbonds.dat` has two columns, frame and count:

```
1 3   2 4   3 4   4 5   5 3   6 4
```

(one pair per line in the real file).

Python 2

```
counts = {}
for line in open("hbonds.dat"):
    frame, n = line.split()
    counts[int(frame)] = int(n)

frames = counts.keys()
frames.sort()
mean = sum(counts.values()) / len(counts)
print "Mean number of H-bonds: %.2f" % mean
print "Frames below the mean:",
for i in xrange(len(frames)):
    if counts[frames[i]] < mean:
        print frames[i],
print
```

```
Mean number of H-bonds: 3.00
Frames below the mean:
```

The script runs without any error in Python 2, and its output looks perfectly reasonable. It is wrong. The counts are integers, so `23 / 6` is 3 instead of 3.83, and since no frame has fewer than 3 hydrogen bonds, the list of frames below the mean is empty.

Running it with Python 3 produces, one after the other, as you fix them:

1. `SyntaxError: Missing parentheses in call to 'print'` (the four `print` statements);
2. `AttributeError: 'dict_keys' object has no attribute 'sort'` (`keys()` is a view);
3. `NameError: name 'xrange' is not defined`.

The division bug produces no error at all: in Python 3, `/` simply gives the right answer. A clean translation, which also uses `with` to close the file and loops directly over the frames:

Python 3

```
counts = {}
with open("hbonds.dat") as f:
    for line in f:
        frame, n = line.split()
        counts[int(frame)] = int(n)

frames = sorted(counts)
mean = sum(counts.values()) / len(counts)
print(f"Mean number of H-bonds: {mean:.2f}")
below = [fr for fr in frames if counts[fr] < mean]
print("Frames below the mean:", *below)
```

```
Mean number of H-bonds: 3.83
Frames below the mean: 1 5
```

> **Lesson.** The errors you see are the easy ones. Before trusting results produced by old code (or by your translation), check it with an input whose answer you know by hand, like this six-line file.

13 Checklist for translating old code

1. **Run it with** `python3` and fix the syntax errors: `print`, `except ..., e`, `<>`, octal and `L` literals.
2. **Search the file** for `xrange`, `raw_input`, `has_key`, `iteritems`, `unicode`, `basestring`, `file(`, `long(` and `reduce(`, and replace them.
3. **Review every** `/` (Section 4). Remove the `float()` and `1.0*` workarounds once you are sure they are no longer needed.
4. **Look for indexing or sorting of** `keys()`, `values()`, `items()`, `zip`, `map` and `filter`, and use `sorted()` or `list()` where a real list is needed.
5. **Check** `round()` if exact rounded values matter for your output.
6. **Give an encoding** when opening text files that may contain non-ASCII characters.
7. **Test** with a case whose answer you know, and compare with independent results, not only with what the old code printed.

> **Tools.** Python versions up to 3.12 include a converter, `2to3`, that applies most of the mechanical changes automatically (it was removed in Python 3.13). It cannot decide what each `/` was meant to do, so step 3 is always your job.

14 Exercise: spot the Python 2

The following function was written for Python 2. Find everything that has to change for it to run in Python 3.

Python 2

```
import string

def read_charges(filename):
    try:
        f = file(filename)
    except IOError, e:
        print "Cannot open", filename
        return {}
    charges = {}
    for line in f.readlines():
        atom, q = string.split(line)
        charges[atom] = float(q)
    f.close()
    return charges

q = read_charges("charges.dat")
if q.has_key("O1") and len(q) <> 0:
    print "Total charge: %.3f" % sum(q.values())
```

The solution is at the end of this document.

15 Further reading

- **What's New In Python 3.0**, the official list of changes:
  https://docs.python.org/3/whatsnew/3.0.html
- **PEP 238**, *Changing the division operator*, which explains why / changed:
  https://peps.python.org/pep-0238/
- **PEP 3105**, *Make print a function*:
  https://peps.python.org/pep-3105/
- **PEP 373**, the Python 2.7 release schedule and end of life:
  https://peps.python.org/pep-0373/
- **Floating-point arithmetic: issues and limitations** (why `round(2.675, 2)` is 2.67):
  https://docs.python.org/3/tutorial/floatingpoint.html

Solution to the exercise

Seven changes: `file()` → `open()`; `except IOError, e` → `except OSError` (`e` was not used); two `print` statements; `string.split(line)` → `line.split()` (and the `import string` is no longer needed); `has_key` → `in`; `<>` → `!=`. Using `with` and an f-string as well:

Python 3

```
def read_charges(filename):
    try:
        with open(filename) as f:
            charges = {}
            for line in f:
                atom, q = line.split()
                charges[atom] = float(q)
    except OSError:
        print("Cannot open", filename)
        return {}
    return charges

q = read_charges("charges.dat")
if "O1" in q and len(q) != 0:
    print(f"Total charge: {sum(q.values()):.3f}")
```

```
Total charge: 0.000
```